Rising Water Project

FLOOD PREDICTION USING MACHINE LEARNING

A Machine Learning-Based Web Application for Flood Risk Prediction Using Weather and Rainfall Data

PROJECT DOCUMENTATION

Submitted by

Raviteja Tulasi

Kodali Shamily

Kalari Lahari

Yalamanchi Mounika

Course: Artificial intelligence & Machine learning

B.Tech – Computer Science and Engineering (Artificial Intelligence)

KKR & KSR Institute of Technology and Sciences

Academic Year 2025–2026

DECLARATION

We hereby declare that the project entitled “Flood Prediction Using Machine Learning” is an original academic work carried out as part of our learning and project development activities. This report presents the analysis, design, machine learning methodology, implementation, testing and deployment of the proposed flood prediction system.

ACKNOWLEDGEMENT

We express our sincere gratitude to the faculty and management of KKR & KSR Institute of Technology and Sciences for providing the academic environment, guidance and resources required to complete this project. We also acknowledge the open-source technologies used in the development of the system, including Python, Flask, Scikit-learn, Pandas, NumPy, XGBoost, GitHub and Render.

ABSTRACT

Floods are among the most destructive natural disasters and can cause serious damage to human life, agriculture, infrastructure and the economy. This project presents a Machine Learning-based Flood Prediction System that analyzes weather and rainfall parameters to estimate flood risk. The system uses ten input features and compares Logistic Regression, Decision Tree, Random Forest and XGBoost. Random Forest was selected for deployment after achieving 95.65% accuracy in the project evaluation. The model is integrated into a Flask web application with registration, login, session management, a personalized dashboard, flood probability estimation and prediction history.

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1 Introduction

Floods are among the most devastating natural disasters, causing significant loss of life, destruction of infrastructure, agricultural damage, and economic setbacks across the world. Every year, millions of people are affected by floods due to excessive rainfall, overflowing rivers, cyclones, and inadequate drainage systems. Climate change has further increased the frequency and intensity of extreme weather events, making flood prediction an essential component of disaster management.

Traditional flood forecasting methods primarily depend on manual observation, hydrological analysis, and historical rainfall records. Although these methods have been useful for many years, they often require extensive computational resources, expert interpretation, and large-scale monitoring systems. Their effectiveness may also decrease when dealing with rapidly changing environmental conditions or localized weather events.

Recent advancements in Artificial Intelligence (AI) and Machine Learning (ML) have introduced new possibilities for flood prediction by identifying complex relationships among environmental variables. Machine learning algorithms can analyze historical weather and rainfall data to recognize patterns associated with flood occurrences and generate accurate predictions. These predictive models can support government agencies, disaster management authorities, researchers, and local communities by providing timely warnings and facilitating informed decision-making.

The **Rising Waters: Machine Learning-Based Flood Prediction System** is designed to predict the likelihood of flood occurrence using supervised machine learning techniques. The system processes multiple meteorological parameters such as temperature, humidity, cloud cover, annual rainfall, and seasonal rainfall measurements to determine whether flood conditions are likely to occur. Various machine learning algorithms, including Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost, are trained and evaluated to identify the most suitable predictive model.

Among these algorithms, the Random Forest Classifier demonstrated the most reliable overall performance and was selected for deployment within the web application. The developed application provides an intuitive interface where authenticated users can enter environmental parameters and instantly obtain flood prediction results. The system also maintains prediction history, user authentication, and dashboard functionality, creating a complete decision-support platform rather than a simple prediction model.

The application is implemented using Python, Flask, Scikit-learn, HTML, CSS, JavaScript, and SQLite. It is deployed on the Render cloud platform, allowing users to access the system through a web browser without installing any additional software.

This project demonstrates the practical integration of machine learning and web technologies to develop a scalable, accessible, and efficient flood prediction system that contributes to disaster preparedness and risk reduction.

1.2 Problem Statement

Floods continue to pose serious challenges to human life, agriculture, transportation, and infrastructure. Existing flood prediction methods often rely on traditional hydrological models that require complex calculations, extensive monitoring equipment, and continuous expert supervision. Such methods may not provide timely predictions for rapidly changing weather conditions and may not be easily accessible to the general public.

There is a growing need for an intelligent, user-friendly, and cost-effective flood prediction system capable of analyzing historical weather data and generating accurate predictions using machine learning techniques. The system should enable users to input weather parameters, obtain immediate flood risk assessments, securely manage user accounts, and maintain prediction records for future analysis.

The objective of this project is to address these challenges by developing a web-based flood prediction system that utilizes machine learning algorithms to improve prediction accuracy while providing an accessible interface for end users.

1.3 Objectives

The primary objectives of this project are:

- To develop an intelligent flood prediction system using machine learning techniques.
- To analyze historical rainfall and weather data for identifying flood-related patterns.
- To preprocess the collected dataset by handling missing values, outliers, and feature preparation.
- To implement and compare multiple supervised learning algorithms including Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost.
- To select the best-performing model based on evaluation metrics.
- To develop a secure web application using the Flask framework.
- To implement user registration and authentication.
- To maintain prediction history using a relational database.
- To deploy the application on the Render cloud platform for public accessibility.

1.4 Scope of the Project

The scope of the Rising Waters project includes the development of a machine learning-based web application capable of predicting flood occurrence based on weather-related parameters. The project encompasses data preprocessing, exploratory data analysis, machine learning model development, model evaluation, web application development, database integration, and cloud deployment.

The application is intended to serve as a decision-support system for educational purposes and preliminary flood risk assessment. It provides authenticated users with prediction capabilities, stores historical prediction records, and offers a responsive user interface suitable for desktop and mobile devices.

Future enhancements may include integration with real-time weather APIs, geographical information systems (GIS), Internet of Things (IoT) sensors, satellite imagery, and automated alert mechanisms to improve prediction accuracy and operational capabilities.

1.5 Applications

The developed system can be applied in several practical scenarios, including:

- Disaster Management Authorities
- Government Flood Monitoring Agencies
- Agricultural Planning
- Urban Development and Drainage Planning
- Educational and Research Institutions
- Environmental Monitoring
- Community Awareness Programs
- Early Warning Support Systems

1.6 Organization of the Report

This report is organized into multiple chapters. Chapter 1 introduces the project, its objectives, scope, and applications. Chapter 2 presents a literature survey on flood prediction and machine learning techniques. Chapter 3 describes the existing and proposed systems along with system requirements. Chapter 4 explains the dataset, exploratory data analysis, and preprocessing methods. Chapter 5 discusses the machine learning models and their evaluation. Chapter 6 covers the system architecture, database design, and implementation details. Chapter 7 presents deployment, testing, and results. Finally, Chapter 8 concludes the project and outlines future enhancements.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

Flood prediction has become an important research area due to the increasing occurrence of floods caused by climate change, rapid urbanization, and changing rainfall patterns. Accurate prediction enables governments, disaster management agencies, and local communities to prepare for potential flood events and minimize the loss of life and property.

Traditional flood forecasting systems rely primarily on hydrological models, river gauge measurements, and meteorological observations. Although these approaches have been widely adopted, they often require extensive computational resources, continuous monitoring, and expert interpretation. Recent advancements in Artificial Intelligence (AI) and Machine Learning (ML) have enabled researchers to develop data-driven flood prediction models capable of learning complex relationships among environmental variables and producing faster, more accurate predictions.

This chapter reviews previous studies on flood prediction techniques, machine learning applications, and identifies the research gap addressed by the proposed system.

2.2 Traditional Flood Prediction Methods

Historically, flood prediction has relied on hydrological and hydraulic models that simulate the movement of water through rivers, reservoirs, and drainage systems. These models use mathematical equations to estimate river discharge, runoff, and flood levels based on rainfall intensity and watershed characteristics.

Common traditional methods include:

- Rainfall-Runoff Models
- River Gauge Monitoring Systems
- Hydraulic Simulation Models
- Statistical Rainfall Analysis
- Weather Forecast-Based Flood Warning Systems

Although these methods provide reliable predictions under controlled conditions, they have several limitations:

- Require extensive historical records
- Depend on specialized monitoring equipment
- Computationally intensive
- Less effective under rapidly changing climatic conditions
- High implementation and maintenance costs

2.3 Machine Learning in Flood Prediction

Machine Learning has emerged as an effective alternative to traditional flood forecasting methods. Unlike conventional mathematical models, machine learning algorithms automatically learn hidden patterns from historical data and make predictions without explicitly programmed rules.

Machine learning techniques offer several advantages:

- Ability to process large datasets efficiently
- Improved prediction accuracy
- Reduced computational complexity after training
- Automatic feature learning
- Adaptability to changing environmental conditions
- Faster prediction time

These characteristics make machine learning highly suitable for disaster management and early warning systems.

2.4 Review of Machine Learning Algorithms

2.4.1 Logistic Regression

Logistic Regression is a supervised classification algorithm widely used for binary classification problems. It estimates the probability that an observation belongs to a particular class using the logistic (sigmoid) function.

Advantages

- Simple implementation
- Easy interpretation
- Fast training
- Effective for linearly separable data

Limitations

- Limited capability for complex nonlinear relationships
- Sensitive to feature correlations

In flood prediction, Logistic Regression serves as a baseline classification model.

2.4.2 Decision Tree

Decision Tree is a supervised learning algorithm that constructs a hierarchical tree structure using decision rules derived from the dataset.

Each internal node represents a feature, while each branch represents a decision outcome.

Advantages

- Easy visualization
- Handles nonlinear data
- Minimal preprocessing required
- Fast prediction

Limitations

- Can overfit the training data
- Sensitive to noisy datasets

Decision Trees are commonly used because they produce human-readable prediction rules.

2.4.3 Random Forest

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees using bootstrap sampling and random feature selection.

Each tree independently predicts the outcome, and the final prediction is determined by majority voting.

Advantages

- High prediction accuracy
- Robust against overfitting
- Handles high-dimensional data
- Resistant to noise
- Excellent generalization ability

Limitations

- Larger model size
- Longer training time than a single Decision Tree

Due to its strong performance, Random Forest is widely used in environmental prediction systems and serves as the deployed model in this project.

2.4.4 K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a non-parametric supervised learning algorithm that classifies new observations based on the majority class among the nearest neighboring data points.

Advantages

- Simple implementation
- No training phase required
- Effective for small datasets
- Suitable for nonlinear classification

Limitations

- Computationally expensive for large datasets
- Sensitive to the choice of K
- Performance affected by irrelevant features

KNN is included in this project to compare its prediction capability with other classification algorithms.

2.4.5 XGBoost

Extreme Gradient Boosting (XGBoost) is an optimized boosting algorithm that builds sequential decision trees to minimize prediction errors.

Advantages

- Excellent predictive performance
- Handles missing values effectively
- Regularization reduces overfitting
- Highly scalable

Limitations

- More complex implementation
- Longer training time
- Hyperparameter tuning required

XGBoost has become one of the most popular algorithms for structured data classification problems.

2.5 Existing Systems

Several flood prediction systems have been proposed by researchers using statistical methods, hydrological simulations, and traditional machine learning models.

However, many existing systems suffer from the following limitations:

- Dependence on manual monitoring
- Limited prediction accuracy
- Lack of user-friendly interfaces
- No cloud deployment
- Absence of prediction history
- No user authentication
- Limited accessibility

2.6 Research Gap

After reviewing existing flood prediction systems, the following research gaps were identified:

- Many systems focus only on prediction without providing a complete web application.
- Few systems include secure user authentication.
- Prediction history is often unavailable.
- Several studies compare only one or two machine learning algorithms.
- Many systems are not deployed online for public access.
- Few educational implementations demonstrate the complete development lifecycle from data preprocessing to deployment.

The proposed project addresses these gaps by integrating machine learning, database management, web development, and cloud deployment into a single platform.

2.7 Proposed Solution

The proposed system, **Rising Waters**, is a web-based flood prediction platform developed using Machine Learning and Flask.

The system:

- Predicts flood occurrence using weather and rainfall parameters.
- Compares five supervised learning algorithms:
    - Logistic Regression
    - Decision Tree
    - Random Forest
    - K-Nearest Neighbors (KNN)
    - XGBoost
- Selects Random Forest as the deployment model based on evaluation results.

- Provides secure user registration and login.
- Stores prediction history using SQLite.
- Displays prediction results through an interactive dashboard.
- Is deployed on the Render cloud platform for online accessibility.

2.8 Summary

This chapter reviewed traditional flood prediction techniques and recent machine learning approaches. Various supervised learning algorithms were examined, highlighting their strengths and limitations. The analysis identified several limitations in existing flood prediction systems, including limited accessibility, lack of authentication, and absence of cloud deployment.

The proposed **Rising Waters** system addresses these shortcomings by combining machine learning, Flask web development, database integration, and cloud deployment to create a practical and user-friendly flood prediction platform.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Introduction

System analysis is a critical phase in software development that involves understanding the problem domain, identifying user requirements, evaluating the feasibility of the proposed solution, and designing an efficient system architecture. In this project, the system analysis focuses on developing an intelligent flood prediction platform capable of providing accurate predictions using machine learning while offering a secure, user-friendly web interface.

The proposed system integrates data preprocessing, machine learning algorithms, database management, and cloud deployment to provide a complete flood prediction solution. Unlike traditional prediction models that focus only on algorithm development, this project emphasizes the development of an end-to-end web application that allows users to register, log in, predict flood occurrence, and maintain prediction history.

3.2 Existing System

Existing flood prediction systems are primarily based on hydrological models, rainfall-runoff simulations, and statistical forecasting techniques. These methods use rainfall records, river water levels, and watershed characteristics to estimate flood occurrence.

While these approaches have been widely adopted, they suffer from several limitations.

Limitations of Existing Systems

- Depend heavily on manual observation and monitoring.
- Require complex hydrological calculations.
- Limited accessibility for general users.
- High computational requirements.
- Lack of intelligent data-driven prediction.
- No secure authentication system.
- No prediction history management.
- Not suitable for cloud deployment in many implementations.
- Limited scalability.

3.3 Proposed System

The proposed system, **Rising Waters**, is a Machine Learning-based Flood Prediction Web Application developed using Flask and Scikit-learn.

The application enables users to register, log in securely, enter weather-related parameters, and receive immediate flood prediction results. Prediction records are stored in a database, allowing users to review previous predictions through a dashboard.

Five machine learning algorithms are implemented and compared:

- Logistic Regression
- Decision Tree
- Random Forest

- K-Nearest Neighbors (KNN)
- XGBoost

Among these models, Random Forest demonstrated the best overall performance and was selected as the deployment model.

The system is deployed on the Render cloud platform, making it accessible through any modern web browser.

3.4 Advantages of the Proposed System

The proposed system offers several advantages over traditional flood prediction methods.

- Intelligent prediction using Machine Learning.
- Higher prediction accuracy.
- Fast response time.
- User-friendly web interface.
- Secure user authentication.
- Cloud-based accessibility.
- Prediction history management.
- Database integration.
- Easy maintenance.
- Scalable architecture.
- Reduced human intervention.

3.5 Functional Requirements

Functional requirements describe the operations that the system must perform.

User Registration

- Allow new users to create accounts.
- Store user credentials securely.
- Prevent duplicate registrations.

User Login

- Authenticate registered users.
- Allow only valid users to access prediction features.

Dashboard

- Display user information.
- Provide navigation to prediction and history pages.

Flood Prediction

- Accept environmental parameters from the user.
- Process the input using the trained Random Forest model.
- Display prediction results.

Prediction History

- Store every prediction in the database.
- Display previous prediction records.
- Allow users to review historical predictions.

Logout

- Securely terminate the user session.

3.6 Non-Functional Requirements

Performance

The application should provide prediction results within a few seconds.

Reliability

The system should consistently generate accurate predictions based on trained machine learning models.

Scalability

The application should support future expansion, including additional datasets and advanced prediction models.

Security

- Password-based authentication.
- Session management.
- Secure database operations.

Availability

The deployed application should remain accessible through cloud hosting.

Maintainability

The modular project structure allows easy updates and future enhancements.

3.7 Feasibility Study

A feasibility study evaluates whether the proposed project can be successfully implemented.

Technical Feasibility

The project uses widely adopted technologies:

- Python
- Flask
- Scikit-learn
- SQLite
- HTML
- CSS
- JavaScript
- Render

These technologies are open-source, stable, and suitable for machine learning web applications.

Economic Feasibility

The project is economically feasible because all software tools and frameworks used are free and open-source.

No expensive hardware or commercial software licenses are required.

Operational Feasibility

The application provides a simple graphical interface that can be used by individuals with basic computer knowledge.

No specialized training is required.

Schedule Feasibility

The project was completed within the planned academic timeline, including:

- Dataset collection
- Data preprocessing
- Model development
- Web application development
- Testing
- Deployment

3.8 Software Requirements

Software	Version/Purpose
Operating System	Windows 10/11 or Linux
Programming Language	Python 3.x
Framework	Flask
Machine Learning Library	Scikit-learn
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Database	SQLite
IDE	Visual Studio Code / Jupyter Notebook
Browser	Google Chrome / Microsoft Edge
Deployment	Render

3.9 Hardware Requirements

Component	Specification
Processor	Intel Core i3 or above
RAM	Minimum 4 GB (8 GB Recommended)
Storage	5 GB Free Space
Internet	Required for Deployment
Display	1366×768 Resolution or Higher

3.10 System Architecture Overview

The proposed system follows a layered architecture consisting of the following components:

Presentation Layer

- HTML
- CSS
- JavaScript

Provides the user interface for registration, login, dashboard, prediction, and history pages.

Application Layer

- Flask Framework

Handles routing, authentication, user sessions, prediction requests, and database communication.

Machine Learning Layer

- Data Preprocessing
- Random Forest Model
- Prediction Engine

Processes user input and generates flood prediction results.

Database Layer

- SQLite Database

Stores user credentials and prediction history.

Deployment Layer

- Render Cloud Platform

Hosts the Flask application, enabling online access.

3.11 Summary

This chapter analyzed the proposed flood prediction system by examining the existing system, identifying its limitations, and presenting an improved solution based on machine learning and web technologies. Functional and non-functional requirements were defined, and the feasibility of the project was evaluated from technical, economic, operational, and scheduling perspectives. The software and hardware requirements, along with the high-level system architecture, establish the foundation for the implementation described in the subsequent chapters.

CHAPTER 4

DATASET DESCRIPTION AND EXPLORATORY DATA ANALYSIS

4.1 Introduction

The performance of any Machine Learning model depends significantly on the quality of the dataset and the preprocessing techniques applied before model training. Data preprocessing and exploratory data analysis (EDA) help identify patterns, inconsistencies, missing values, and relationships among variables, ultimately improving the prediction capability of the model.

In this project, historical weather and rainfall data were collected and analyzed to build an intelligent flood prediction model. The dataset consists entirely of numerical attributes representing meteorological conditions and seasonal rainfall measurements. These features serve as the input variables for training multiple supervised machine learning algorithms.

4.2 Dataset Description

The dataset used in this project contains historical weather observations and rainfall measurements collected from reliable meteorological sources. Each record represents environmental conditions corresponding to a particular observation, while the target variable indicates whether flood conditions occurred.

The dataset is organized in tabular form, where each row represents one observation and each column represents a specific environmental feature.

The target variable is binary:

- 0 – No Flood
- 1 – Flood

4.3 Dataset Features

Feature	Description
Temperature	Average atmospheric temperature
Humidity	Relative humidity percentage
Cloud Cover	Percentage of cloud cover
Annual Rainfall	Total annual rainfall
Jan-Feb Rainfall	Rainfall during January–February
Mar-May Rainfall	Rainfall during March–May
Jun-Sep Rainfall	Rainfall during June–September
Oct-Dec Rainfall	Rainfall during October–December
Average June Rainfall	Average rainfall during June
Sub Rainfall	Additional rainfall indicator used in the dataset

Target Variable

Flood

0 – No Flood

1 – Flood

4.4 Dataset Loading

The dataset is imported into Python using the Pandas library. It is stored as a DataFrame, which provides efficient data manipulation and analysis capabilities. After loading, the dataset is examined to verify that all records and columns have been imported correctly.

4.5 Dataset Information

The structure of the dataset is analyzed to determine the number of records, number of attributes, data types, memory usage, and presence of missing values. Understanding the dataset structure is essential before applying preprocessing and machine learning algorithms.

4.6 Descriptive Statistics

Descriptive statistical analysis is performed to summarize the numerical characteristics of the dataset. The statistical measures include count, mean, standard deviation, minimum value, maximum value, first quartile, median, and third quartile. These statistics provide insights into the distribution and variability of each feature.

4.7 Missing Value Analysis

The dataset is examined for missing or null values before model training. Missing values can significantly affect prediction accuracy and therefore must be identified during preprocessing.

After analysis, it was observed that the dataset does not contain significant missing values. Therefore, no missing value imputation techniques were required.

4.8 Duplicate Record Analysis

Duplicate records may introduce bias into machine learning models and reduce prediction accuracy. The dataset is checked for duplicate entries, and any duplicate records identified are removed before model training to improve data quality.

4.9 Handling Categorical Values

The dataset was examined to identify categorical attributes. The analysis confirmed that all features are numerical in nature and no categorical columns are present.

Therefore, Label Encoding and One-Hot Encoding were not required. The numerical attributes were directly used for machine learning model training.

4.10 Univariate Analysis

Univariate Analysis examines each feature independently to understand its statistical distribution. Histograms, density plots, and box plots are used to study central tendency, spread, skewness, and potential outliers.

This analysis provides a better understanding of the individual characteristics of each environmental parameter.

4.11 Multivariate Analysis

Multivariate Analysis investigates relationships among multiple variables. Correlation analysis is performed to determine how rainfall and weather parameters influence flood occurrence.

A correlation heatmap is generated to visualize the strength and direction of relationships among all features. This helps identify the most influential variables for flood prediction.

4.12 Outlier Analysis

Outliers are extreme observations that may influence machine learning performance. Box plots are used to identify unusually high or low values within the dataset.

The observed outliers represent realistic environmental conditions rather than data entry errors. Therefore, they were retained to preserve the natural characteristics of the dataset.

4.13 Feature Scaling

Feature scaling standardizes numerical variables so that they have comparable ranges. Standardization improves the performance of several machine learning algorithms by preventing features with larger values from dominating the learning process.

A StandardScaler is used where applicable to normalize the feature values before model training. The fitted scaler is saved for use during prediction in the deployed application.

4.14 Feature Selection

The dataset is divided into independent variables and the target variable.

Independent Variables:

- Temperature
- Humidity
- Cloud Cover
- Annual Rainfall
- Jan-Feb Rainfall
- Mar-May Rainfall
- Jun-Sep Rainfall
- Oct-Dec Rainfall
- Average June Rainfall
- Sub Rainfall

Dependent Variable:

Flood

The selected features are used to train and evaluate multiple supervised machine learning models.

4.15 Train-Test Split

The processed dataset is divided into training and testing datasets using an 80:20 ratio.

The training dataset is used to build the machine learning models, while the testing dataset is used to evaluate prediction performance on unseen data.
This approach helps estimate the model's ability to generalize to new observations.

4.16 Summary

This chapter described the dataset used for flood prediction and the preprocessing techniques applied before model development. Exploratory Data Analysis was performed to understand the dataset characteristics through descriptive statistics, missing value analysis, duplicate record detection, univariate analysis, multivariate analysis, and outlier analysis.
The dataset contains only numerical features and therefore does not require categorical encoding. After preprocessing, the dataset was divided into training and testing sets, preparing it for the implementation and evaluation of multiple machine learning algorithms.
The next chapter presents the implementation of Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost models along with their performance comparison and model selection.

CHAPTER 5

MACHINE LEARNING MODEL DEVELOPMENT AND EVALUATION

5.1 Introduction

Machine Learning is a branch of Artificial Intelligence that enables computer systems to learn patterns from historical data and make predictions without being explicitly programmed for every scenario. In this project, supervised machine learning techniques are used to predict flood occurrence based on historical weather and rainfall parameters.

Several classification algorithms were implemented and evaluated to determine the most suitable model for flood prediction. The performance of each model was assessed using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix. Based on the comparative analysis, the Random Forest classifier demonstrated the best overall performance and was selected as the final prediction model for deployment.

5.2 Machine Learning Workflow

The machine learning workflow adopted in this project consists of the following stages:

- Dataset Collection
- Data Preprocessing
- Exploratory Data Analysis
- Feature Selection
- Train-Test Split
- Model Training
- Model Evaluation
- Model Comparison
- Model Selection
- Model Saving
- Deployment

This workflow ensures that the developed prediction model is accurate, reliable, and suitable for real-world deployment.

5.3 Logistic Regression

Logistic Regression is a supervised machine learning algorithm used for binary classification problems. It predicts the probability of a particular class using the sigmoid function and classifies the output into one of two categories.

In this project, Logistic Regression was trained using the processed dataset and evaluated on the testing dataset.

Advantages

- Simple and easy to implement.
- Fast training and prediction.
- Performs well for linearly separable data.
- Easy to interpret.

Limitations

• Performs poorly on highly nonlinear datasets.
• Sensitive to correlated features.
• Lower accuracy compared to ensemble models.

5.4 Decision Tree Classifier

Decision Tree is a supervised learning algorithm that creates a hierarchical tree-like structure based on decision rules. The algorithm recursively splits the dataset into smaller subsets until the stopping criteria are satisfied.

Each internal node represents a decision, while each leaf node represents the predicted class.

Advantages

• Easy to understand and visualize.
• Handles nonlinear relationships.
• Requires minimal data preprocessing.
• Fast prediction.

Limitations

• Can overfit the training data.
• Sensitive to small variations in the dataset.
• Lower generalization capability compared to ensemble methods.

5.5 Random Forest Classifier

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees to improve prediction accuracy and reduce overfitting. Each tree is trained using a randomly selected subset of data and features.

The final prediction is obtained through majority voting among all decision trees.

Random Forest produced the highest prediction performance among the implemented algorithms and was therefore selected for deployment.

Advantages

• High prediction accuracy.
• Resistant to overfitting.
• Handles noisy datasets effectively.
• Robust against missing values.
• Excellent generalization capability.

Limitations

• Larger model size.
• Slightly longer training time.
• Less interpretable than a single Decision Tree.

5.6 K-Nearest Neighbors (KNN)

K-Nearest Neighbors is a non-parametric supervised learning algorithm that classifies an observation

based on the majority class among its nearest neighboring data points.

The algorithm calculates the distance between the input sample and existing training samples before assigning the final class.

Advantages

- Easy to understand.
- No explicit training phase.
- Effective for smaller datasets.
- Suitable for nonlinear classification.

Limitations

- Computationally expensive during prediction.
- Sensitive to the value of K.
- Performance decreases for very large datasets.

KNN was included in this project to compare its performance with other supervised classification algorithms.

5.7 XGBoost Classifier

Extreme Gradient Boosting (XGBoost) is an advanced boosting algorithm that constructs multiple decision trees sequentially while minimizing prediction errors.

The algorithm applies gradient boosting and regularization techniques to improve prediction accuracy while reducing overfitting.

Advantages

- High predictive accuracy.
- Excellent handling of structured datasets.
- Supports regularization.
- Robust against overfitting.

Limitations

- More complex implementation.
- Requires hyperparameter tuning.
- Higher computational cost.

5.8 Model Training

Each machine learning model was trained using the training dataset obtained after preprocessing.

The workflow involved:

- Loading the processed dataset.
- Selecting input and output variables.
- Splitting the dataset into training and testing sets.
- Training each classification algorithm.
- Generating predictions on the testing dataset.
- Evaluating prediction performance.

The same dataset was used for all algorithms to ensure a fair comparison.

5.9 Performance Evaluation Metrics

The performance of each classification model was evaluated using standard classification metrics.

Accuracy

Accuracy represents the percentage of correctly classified observations.

Precision

Precision measures the proportion of positive predictions that are actually correct.

Recall

Recall indicates the ability of the model to correctly identify flood occurrences.

F1-Score

The F1-Score represents the harmonic mean of Precision and Recall.

Confusion Matrix

The Confusion Matrix provides detailed information regarding correctly and incorrectly classified observations.

These metrics collectively provide a comprehensive evaluation of the prediction capability of each machine learning algorithm.

5.10 Model Comparison

The prediction accuracy of all implemented machine learning algorithms was compared to determine the best-performing model.

The implemented models include:

- Logistic Regression
- Decision Tree
- Random Forest
- K-Nearest Neighbors (KNN)
- XGBoost

A comparison table and graphical visualization were prepared to compare the performance of all algorithms.

5.11 Model Selection

Based on the experimental evaluation, Random Forest demonstrated the most reliable and consistent prediction performance.

The reasons for selecting Random Forest include:

• Higher prediction accuracy.
• Better handling of nonlinear relationships.
• Reduced overfitting.
• Strong generalization capability.
• Stable prediction performance on unseen data.

Therefore, the Random Forest classifier was selected as the final prediction model for deployment within the Flask web application.

5.12 Model Saving

After selecting the best-performing model, the trained Random Forest classifier was saved using the Joblib library.

The saved model allows the Flask application to generate predictions without retraining the model each time the application starts.

Similarly, the preprocessing transformer was also saved to ensure consistency between training and prediction phases.

5.13 Summary

This chapter discussed the development and evaluation of multiple supervised machine learning algorithms for flood prediction. Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost were trained and evaluated using the prepared dataset. Their prediction performances were compared using standard classification metrics.

Among all the implemented models, Random Forest demonstrated superior overall performance and was selected as the final deployment model. The trained model and preprocessing objects were saved for integration into the Flask-based web application.

The next chapter presents the implementation of the web application, including system architecture, database design, user authentication, prediction module, dashboard, and deployment.

CHAPTER 6

SYSTEM DESIGN AND IMPLEMENTATION

6.1 Introduction

The implementation phase transforms the designed machine learning model into a fully functional web application. The Rising Waters Flood Prediction System integrates machine learning, database management, and web technologies to provide an intelligent platform for flood prediction. The system allows users to register, log in securely, predict flood occurrence using weather parameters, and store prediction history for future reference.

The application follows a modular architecture that separates the user interface, application logic, machine learning model, and database, making the system scalable, maintainable, and easy to extend.

6.2 System Architecture

The proposed system follows a layered architecture consisting of four major layers:

Presentation Layer

The presentation layer provides the graphical user interface through which users interact with the application. It includes web pages developed using HTML, CSS, and JavaScript.

Functions include:

- User Registration
- User Login
- Dashboard
- Prediction Form
- Prediction History
- Logout

Application Layer

The application layer is developed using the Flask web framework. It processes user requests, validates input, communicates with the machine learning model, manages user sessions, and stores prediction records.

Functions include:

- URL Routing
- Session Management
- Authentication
- Form Validation
- Prediction Processing
- Database Communication

Machine Learning Layer

This layer contains the trained machine learning model responsible for generating flood predictions.

Components include:

- Trained Random Forest Model
- Data Preprocessing
- Feature Transformation
- Prediction Engine

Database Layer

SQLite is used to store application data.

The database stores:

- User Information
- Login Credentials
- Prediction Records
- Prediction Date and Time

6.3 System Workflow

The complete workflow of the proposed system is as follows:

Step 1

The user opens the web application.

Step 2

The user registers a new account or logs into an existing account.

Step 3

After successful authentication, the dashboard is displayed.

Step 4

The user enters weather and rainfall parameters.

Step 5

The input values are validated.

Step 6

The trained Random Forest model processes the input data.

Step 7

The prediction result is generated.

Step 8

The prediction result is displayed.

Step 9

The prediction details are stored in the SQLite database.

Step 10

The user can view previous predictions through the Prediction History page.

6.4 Database Design

SQLite is used as the relational database management system because it is lightweight, efficient, and easy to integrate with Flask applications.

The database stores user information and prediction history.

The database improves system usability by maintaining historical prediction records for authenticated users.

6.5 Entity Relationship (ER) Diagram

The database contains the following primary entities:

User

Attributes:

• User ID
• Name
• Email
• Password

Prediction

Attributes:

• Prediction ID
• User ID
• Temperature
• Humidity
• Cloud Cover
• Annual Rainfall
• Jan-Feb Rainfall
• Mar-May Rainfall
• Jun-Sep Rainfall
• Oct-Dec Rainfall
• Average June Rainfall
• Sub Rainfall
• Prediction Result
• Prediction Date

Relationship

One User can have multiple Prediction records.

This represents a One-to-Many relationship.

6.6 User Authentication Module

The authentication module ensures that only registered users can access the flood prediction system.

Functions include:

• User Registration
• Login
• Password Verification
• Session Creation
• Logout

Authentication prevents unauthorized access to prediction history and dashboard features.

6.7 Dashboard Module

The dashboard serves as the central interface after successful login.

The dashboard provides navigation to:

- Flood Prediction
- Prediction History
- User Profile
- Logout

The dashboard improves usability by providing quick access to all major system functionalities.

6.8 Prediction Module

The Prediction Module is the core component of the application.

Users enter the following parameters:

- Temperature
- Humidity
- Cloud Cover
- Annual Rainfall
- Jan-Feb Rainfall
- Mar-May Rainfall
- Jun-Sep Rainfall
- Oct-Dec Rainfall
- Average June Rainfall
- Sub Rainfall

After clicking the Predict button, the application processes the input using the trained Random Forest model.

The prediction result is displayed as either:

Flood Likely

or

No Flood Likely

6.9 Prediction History Module

Every prediction generated by the application is stored in the SQLite database.

The Prediction History page allows users to:

- View previous predictions
- Review historical weather inputs
- Compare prediction results
- Maintain a prediction log

This functionality improves the usability of the application and allows users to monitor previous predictions.

6.10 Front-End Design

The user interface was developed using standard web technologies.

Technologies used:

- HTML5
- CSS3
- JavaScript

The interface was designed to be simple, responsive, and user-friendly.

Separate pages were developed for:

- Home
- Register
- Login
- Dashboard
- Prediction
- Prediction Result
- No Flood Result
- Prediction History

6.11 Back-End Implementation

The back-end of the application was implemented using the Flask framework.

Major responsibilities include:

- Routing
- Form Handling
- User Authentication
- Machine Learning Prediction
- Database Operations
- Session Management

Flask acts as the communication bridge between the user interface, machine learning model, and SQLite database.

6.12 Project Structure

The project follows a modular folder structure.

Main folders include:

- templates/
- static/
- models/
- database/
- notebook/
- app.py
- requirements.txt
- Procfile

This modular organization improves maintainability and future scalability.

6.13 Deployment

The completed Flask application was deployed using the Render cloud platform.

Deployment steps included:

- Uploading the source code to GitHub.
- Creating a Render Web Service.
- Installing project dependencies using requirements.txt.
- Loading the trained Random Forest model.
- Configuring the Procfile.
- Deploying the Flask application.

Cloud deployment enables users to access the system through any modern web browser without installing additional software.

6.14 Summary

This chapter presented the design and implementation of the Rising Waters Flood Prediction System. The application integrates Flask, SQLite, HTML, CSS, JavaScript, and a trained Random Forest machine learning model to create a complete web-based prediction platform. User authentication, prediction history, dashboard functionality, and cloud deployment enhance the usability and reliability of the system.

The next chapter presents system testing, experimental results, model evaluation, application screenshots, and performance analysis.

CHAPTER 7

SYSTEM TESTING AND RESULTS

7.1 Introduction

Testing is an essential phase of software development that ensures the developed application performs according to the specified requirements. It verifies the correctness, reliability, usability, and efficiency of the system before deployment.

In this project, both the machine learning model and the Flask web application were thoroughly tested. The testing process involved validating user authentication, flood prediction functionality, database operations, navigation, and overall system performance. The prediction accuracy of various machine learning algorithms was also evaluated to identify the most suitable model for deployment.

7.2 Testing Objectives

The objectives of system testing are:

- To verify that all modules function correctly.
- To validate the prediction accuracy of the machine learning model.
- To ensure proper user authentication.
- To verify database operations.

• To evaluate the usability of the web application.
• To identify and eliminate software defects.
• To ensure the application performs reliably under normal operating conditions.

7.3 Types of Testing Performed

Unit Testing

Unit testing was performed on individual modules such as registration, login, prediction, and database operations to verify that each component functions independently.

Integration Testing

Integration testing ensured that different modules communicate correctly. The interaction between the Flask application, machine learning model, and SQLite database was verified.

System Testing

The complete application was tested as an integrated system to ensure all functionalities operate together without errors.

Functional Testing

Functional testing verified that every feature works according to the specified requirements.

Features tested include:

• User Registration
• User Login
• Dashboard Navigation
• Flood Prediction
• Prediction History
• Logout

Performance Testing

The prediction response time and application performance were evaluated to ensure smooth operation under normal usage.

User Acceptance Testing

The application was tested from the end-user perspective to ensure it is easy to use, responsive, and produces accurate prediction results.

7.4 Test Cases

Test Case 1

Module: User Registration

Objective:
Verify successful registration of a new user.

Input:
Valid user information.

Expected Result:
User account is created successfully.

Actual Result:
Successfully registered.

Status:
Pass

Test Case 2

Module: User Login

Objective:
Verify login using valid credentials.

Input:
Registered email and password.

Expected Result:
Dashboard is displayed.

Actual Result:
Login successful.

Status:
Pass

Test Case 3

Module: Invalid Login

Objective:
Verify login with incorrect credentials.

Input:
Invalid email or password.

Expected Result:
Login should fail with an error message.

Actual Result:
Appropriate error message displayed.

Status:
Pass

Test Case 4

Module: Flood Prediction

Objective:
Verify prediction using valid weather parameters.

Input:
Temperature, Humidity, Cloud Cover, Rainfall values.

Expected Result:
Flood prediction generated successfully.

Actual Result:
Prediction displayed correctly.

Status:
Pass

Test Case 5

Module: Prediction History

Objective:
Verify storage of prediction records.

Input:
Prediction generated successfully.

Expected Result:
Prediction saved in database.

Actual Result:
Prediction history updated successfully.

Status:
Pass

Test Case 6

Module: Logout

Objective:
Verify user logout functionality.

Input:
Click Logout.

Expected Result:
User session terminated.

Actual Result:
Successfully redirected to Login page.

Status:
Pass

7.5 Model Performance

Five supervised machine learning algorithms were implemented and evaluated.

The evaluated algorithms include:

- Logistic Regression
- Decision Tree
- Random Forest
- K-Nearest Neighbors (KNN)
- XGBoost

The models were trained using the same dataset and evaluated using the testing dataset.

The comparison was performed using Accuracy, Precision, Recall, and F1-Score.

Based on the experimental results, Random Forest achieved the best overall performance and was selected as the final deployment model.

7.6 Performance Comparison

The performance comparison of all implemented machine learning algorithms demonstrates that ensemble learning techniques outperform individual classifiers for this dataset.

Random Forest produced the highest prediction accuracy and the most stable performance across all evaluation metrics.

The comparison graph and accuracy table clearly indicate the superiority of the Random Forest classifier for flood prediction.

7.7 Experimental Results

The deployed application successfully predicts flood occurrence based on user-provided environmental parameters.

The application provides the following functionality:

- Secure user authentication.
- Accurate flood prediction.
- Fast prediction response.
- Prediction history management.
- Cloud accessibility through Render.

All application modules performed according to the specified functional requirements.

7.8 Advantages of the Developed System

The developed system provides several advantages over traditional flood prediction methods.

- High prediction accuracy.
- Fast response time.
- User-friendly interface.

• Secure authentication system.
• Cloud-based deployment.
• Prediction history storage.
• Low maintenance cost.
• Easy scalability.
• Reliable machine learning prediction.
• Efficient database management.

7.9 Limitations

Although the developed system performs effectively, certain limitations remain.

• Prediction accuracy depends on the quality of the dataset.
• Limited environmental parameters are considered.
• The model is trained using historical data only.
• Real-time weather APIs are not integrated.
• The application currently supports only binary flood prediction.

These limitations provide opportunities for future enhancement.

7.10 Future Scope

Several improvements can be incorporated into future versions of the project.

• Integration with real-time weather APIs.
• Support for satellite imagery analysis.
• IoT sensor integration.
• Mobile application development.
• Multi-language support.
• Flood severity prediction.
• GIS-based flood mapping.
• SMS and Email alert systems.
• Deep Learning-based prediction models.
• Integration with government disaster management systems.

7.11 Summary

This chapter presented the testing procedures and performance evaluation of the Rising Waters Flood Prediction System. Various testing techniques, including unit testing, integration testing, system testing, functional testing, and user acceptance testing, were conducted to verify the correctness and reliability of the application.

The machine learning models were evaluated using standard performance metrics, and Random Forest demonstrated the highest prediction accuracy, making it the most suitable model for deployment. The successful implementation and testing confirm that the developed system meets its objectives and provides an effective solution for flood prediction.

The next chapter presents the conclusion, overall project achievements, limitations, and future enhancements.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Introduction

The final phase of any software development project involves evaluating the overall achievements of the developed system and identifying opportunities for future improvements. This chapter presents the conclusion of the "Rising Waters: Machine Learning-Based Flood Prediction System" by summarizing the work carried out, discussing the outcomes achieved, highlighting the limitations of the current implementation, and suggesting possible enhancements for future development.

8.2 Conclusion

Floods are among the most destructive natural disasters, causing significant damage to human life, agriculture, infrastructure, and the economy. Accurate and timely flood prediction is therefore essential for minimizing losses and improving disaster preparedness. Traditional flood prediction methods often require extensive manual analysis and complex hydrological models, making them less accessible and time-consuming.

The primary objective of this project was to develop a machine learning-based web application capable of predicting the likelihood of floods using historical weather and rainfall data. The system successfully integrates multiple machine learning algorithms, including Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost, to evaluate their prediction performance. Based on the comparative analysis, the Random Forest classifier achieved the best overall performance and was selected as the final deployment model.

The developed application provides a complete end-to-end solution by combining machine learning with web technologies. Users can securely register and log into the system, enter weather-related parameters, receive instant flood predictions, and maintain a history of previous prediction records. The Flask framework was used to develop the backend, HTML, CSS, and JavaScript were used for the frontend, SQLite was used for database management, and the application was successfully deployed on the Render cloud platform.

The project demonstrates that machine learning techniques can effectively improve flood prediction accuracy while providing an easy-to-use interface for end users. The integration of secure authentication, prediction history management, and cloud deployment enhances the practicality and usability of the system. Overall, the project successfully meets its objectives and provides a reliable decision-support tool for flood prediction based on historical environmental data.

8.3 Project Achievements

The major achievements of the project are listed below.

• Successfully developed a machine learning-based flood prediction system.

• Implemented and compared five supervised machine learning algorithms.

• Selected Random Forest as the best-performing prediction model.

• Developed a responsive web application using the Flask framework.

• Implemented secure user registration and login functionality.

• Designed an interactive dashboard for easy navigation.

• Integrated SQLite database for storing user information and prediction history.

• Enabled users to view previous prediction records.

• Successfully deployed the application on the Render cloud platform.

• Created a scalable and modular project architecture suitable for future enhancements.

8.4 Limitations

Although the developed system performs effectively, certain limitations exist.

• Prediction accuracy depends on the quality and size of the training dataset.

• The model considers only the available weather and rainfall parameters.

• Real-time weather information is not integrated into the current system.

• The application predicts only binary outcomes (Flood or No Flood).

• The system does not currently estimate flood severity or affected geographical regions.

• Prediction results may vary if environmental conditions differ significantly from the training dataset.

These limitations provide opportunities for further research and future improvements.

8.5 Future Scope

The proposed system can be extended by incorporating several advanced features to improve prediction accuracy and usability.

• Integration with real-time weather APIs for live environmental data.

• Incorporation of satellite imagery and remote sensing data.

• Integration with IoT-based rainfall and water-level sensors.

• Development of Android and iOS mobile applications.

• Implementation of Deep Learning models such as Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), and Long Short-Term Memory (LSTM) networks.

• Addition of flood severity prediction instead of binary classification.

• GIS-based flood mapping for visual representation of flood-prone areas.

• Automatic SMS and Email notification systems for early flood warnings.

• Multi-language support to improve accessibility for diverse users.

• Integration with government disaster management and emergency response systems.

• Expansion of the dataset to include additional environmental and geographical parameters for improved prediction performance.

8.6 Overall Outcome

The Rising Waters Flood Prediction System demonstrates how machine learning can be effectively applied to solve real-world environmental challenges. The integration of predictive analytics, web technologies, and cloud deployment provides an efficient and user-friendly platform for flood prediction.

The developed system offers accurate predictions, secure user management, historical record maintenance, and cloud accessibility, making it suitable for educational purposes and as a foundation for future research in disaster management. With additional enhancements such as real-time data integration and advanced predictive models, the system can evolve into a comprehensive flood early warning solution capable of supporting disaster preparedness and risk reduction initiatives.

REFERENCES

[1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville, "Deep Learning", MIT Press, 2016.

[2] Aurélien Géron, "Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow", O'Reilly Media, 3rd Edition, 2022.

[3] Christopher M. Bishop, "Pattern Recognition and Machine Learning", Springer, 2006.

[4] Tom M. Mitchell, "Machine Learning", McGraw-Hill Education, 1997.

[5] Scikit-learn Developers, "Scikit-learn Documentation."

[6] Flask Documentation.

[7] Pandas Documentation.

[8] NumPy Documentation.

[9] Matplotlib Documentation.

[10] XGBoost Documentation.

[11] Python Software Foundation, "Python Documentation."

[12] SQLite Documentation.

[13] Render Documentation.

[14] National Disaster Management Authority (NDMA), India.

[15] India Meteorological Department (IMD).

APPENDIX

Appendix A

PROJECT FOLDER STRUCTURE

```
Rising-Waters/
│
├── app.py
├── requirements.txt
├── Procfile
├── README.md
│
├── models/
│   ├── floods.save
│   └── transform.save
│
├── data/
│   ├── flood_prediction.xlsx
│   └── rainfall in india 1901-2015.xlsx
│
├── templates/
│   ├── home.html
│   ├── login.html
│   ├── register.html
│   ├── dashboard.html
│   ├── history.html
│   ├── chance.html
│   └── no_chance.html
│
├── static/
│   ├── css/
│   ├── images/
│   └── js/
│
└── notebook/
    └── Flood_Prediction.ipynb
```

Appendix B

SAMPLE INPUT FOR FLOOD PREDICTION

Temperature	28
Humidity	75
Cloud Cover	40
Annual Rainfall	3326.6
Jan-Feb Rainfall	9.3
Mar-May Rainfall	275.7
Jun-Sep Rainfall	2403.4
Oct-Dec Rainfall	638.2
Average June Rainfall	130.3
Sub Rainfall	256.4

Prediction Result
Flood Likely

Appendix C

MACHINE LEARNING MODELS IMPLEMENTED

1. Logistic Regression
2. Decision Tree
3. Random Forest
4. K-Nearest Neighbors (KNN)
5. XGBoost

Final Selected Model:
Random Forest

Appendix D

TECHNOLOGIES USED

Programming Language
• Python

Frontend
• HTML5
• CSS3
• JavaScript

Backend
• Flask

Machine Learning Libraries
• Scikit-learn
• Pandas
• NumPy
• Matplotlib

Database
• SQLite

Development Tools
• Visual Studio Code
• Jupyter Notebook

Version Control
• Git
• GitHub

Deployment Platform
• Render

Appendix E

SOFTWARE REQUIREMENTS
Operating System : Windows 10/11 or Linux
Processor : Intel Core i3 or Higher
RAM : Minimum 4 GB
Storage : 5 GB Free Space
Internet Connection : Required
Browser : Google Chrome / Microsoft Edge
Python : Version 3.14

Appendix F

MODEL PERFORMANCE

Algorithm	Accuracy (%)
Logistic Regression	91.30
Decision Tree	95.65
Random Forest	95.65
K-Nearest Neighbors	86.96
XGBoost	86.96

Final Model Selected:
Random Forest

Appendix G

GITHUB REPOSITORY

Repository Name:
Rising-Waters

GitHub Link:
https://github.com/RavitejaTulasi/Rising-water

Appendix H

DEPLOYMENT LINK

Application Name:
Rising Waters: Machine Learning-Based Flood Prediction System

Deployment URL:
https://rising-water-l3ro.onrender.com

Appendix I

USER MANUAL

Step 1:
Open the web application.

Step 2:
Register a new account.

Step 3:
Login using registered credentials.

Step 4:
Enter the required weather parameters.

Step 5:
Click the Predict button.

Step 6:
View the prediction result.

Step 7:
Access the Prediction History page to view previous predictions.

Step 8:
Logout securely after completing the session.